

ALU UVM Verification

Generated by Doxygen 1.16.1

1 Topic Index	1
1.1 Topics	1
2 Directory Hierarchy	3
2.1 Directories	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Topic Documentation	11
6.1 RTL design	11
6.1.1 Detailed Description	11
6.2 Modules	11
6.3 Testbench components	11
6.3.1 Detailed Description	11
6.4 Interfaces	12
6.4.1 Detailed Description	12
6.4.2 Function Documentation	12
6.4.2.1 s_interface()	12
6.5 UVM verification components	12
6.5.1 Detailed Description	13
7 Directory Documentation	15
7.1 filter Directory Reference	15
7.1.1 Detailed Description	15
7.1.2 SystemVerilog Doxygen filter	15
7.2 reference_model Directory Reference	15
7.3 rtl Directory Reference	15
7.4 tb Directory Reference	16
7.5 uvm Directory Reference	16
8 Class Documentation	17
8.1 agent Class Reference	17
8.1.1 Detailed Description	18
8.1.2 Member Function Documentation	18
8.1.2.1 build_phase()	18
8.1.2.2 connect_phase()	18
8.1.2.3 new()	18
8.1.2.4 uvm_component_utils()	19

8.1.3 Member Data Documentation	19
8.1.3.1 m_monitor	19
8.1.3.2 m_sequencer	19
8.2 alu_sequence Class Reference	19
8.2.1 Detailed Description	20
8.2.2 Member Function Documentation	20
8.2.2.1 body()	20
8.2.2.2 new()	20
8.2.2.3 uvm_object_utils()	21
8.2.3 Member Data Documentation	21
8.2.3.1 num_packets	21
8.3 coverage Class Reference	21
8.3.1 Detailed Description	22
8.3.2 Member Function Documentation	22
8.3.2.1 cg_alu()	22
8.3.2.2 new()	22
8.3.2.3 uvm_component_utils()	22
8.3.2.4 write()	23
8.4 driver Class Reference	23
8.4.1 Detailed Description	24
8.4.2 Member Function Documentation	24
8.4.2.1 build_phase()	24
8.4.2.2 drive_item()	24
8.4.2.3 new()	24
8.4.2.4 run_phase()	25
8.4.2.5 uvm_component_utils()	25
8.4.3 Member Data Documentation	25
8.4.3.1 req_pkt	25
8.5 env Class Reference	25
8.5.1 Detailed Description	26
8.5.2 Member Function Documentation	26
8.5.2.1 build_phase()	26
8.5.2.2 connect_phase()	27
8.5.2.3 new()	27
8.5.2.4 uvm_component_utils()	27
8.5.3 Member Data Documentation	27
8.5.3.1 m_coverage	27
8.5.3.2 m_scoreboard	27
8.6 monitor Class Reference	28
8.6.1 Detailed Description	29
8.6.2 Member Function Documentation	29
8.6.2.1 build_phase()	29

8.6.2.2 new()	29
8.6.2.3 run_phase()	29
8.6.2.4 uvm_component_utils()	29
8.6.3 Member Data Documentation	29
8.6.3.1 mon_analysis_port	29
8.7 pkt Class Reference	30
8.7.1 Detailed Description	31
8.7.2 Member Function Documentation	31
8.7.2.1 alu_ctrl_range()	31
8.7.2.2 new()	31
8.7.2.3 uvm_field_int() [1/2]	31
8.7.2.4 uvm_field_int() [2/2]	31
8.7.2.5 uvm_object_utils_begin()	31
8.7.3 Member Data Documentation	32
8.7.3.1 i_alu_ctrl_EX	32
8.7.3.2 i_rd1_EX	32
8.7.3.3 i_rd2_EX	32
8.7.3.4 o_alu_result_EX	32
8.7.3.5 o_equal_EX	32
8.8 s_interface Interface Reference	32
8.8.1 Detailed Description	32
8.9 scoreboard Class Reference	33
8.9.1 Detailed Description	34
8.9.2 Member Function Documentation	34
8.9.2.1 build_phase()	34
8.9.2.2 check_phase()	34
8.9.2.3 new()	34
8.9.2.4 run_phase()	34
8.9.2.5 uvm_component_utils()	34
8.9.2.6 write()	35
8.9.3 Member Data Documentation	35
8.9.3.1 ap_imp	35
8.9.3.2 num_errors	35
8.10 sequencer Class Reference	35
8.10.1 Detailed Description	36
8.10.2 Member Function Documentation	36
8.10.2.1 uvm_component_utils()	36
8.10.3 Member Data Documentation	36
8.10.3.1 parent	36
8.11 test Class Reference	37
8.11.1 Detailed Description	38
8.11.2 Member Function Documentation	38

8.11.2.1 build_phase()	38
8.11.2.2 end_of_elaboration_phase()	38
8.11.2.3 new()	38
8.11.2.4 run_phase()	38
8.11.2.5 uvm_component_utils()	38
8.11.3 Member Data Documentation	38
8.11.3.1 m_sequence	38
9 File Documentation	39
9.1 filter/README.md File Reference	39
9.2 reference_model/reference_model.c File Reference	39
9.2.1 Detailed Description	40
9.2.2 Macro Definition Documentation	40
9.2.2.1 OP_ADD	40
9.2.2.2 OP_AND	40
9.2.2.3 OP_AUIPC	40
9.2.2.4 OP_BEQ	40
9.2.2.5 OP_BGE	41
9.2.2.6 OP_BGEU	41
9.2.2.7 OP_BLT	41
9.2.2.8 OP_BLTU	41
9.2.2.9 OP_BNE	41
9.2.2.10 OP_EBREAK	41
9.2.2.11 OP_ECALL	41
9.2.2.12 OP_FENCE	41
9.2.2.13 OP_LUI	41
9.2.2.14 OP_OR	41
9.2.2.15 OP_SLL	42
9.2.2.16 OP_SLT	42
9.2.2.17 OP_SLTU	42
9.2.2.18 OP_SRA	42
9.2.2.19 OP_SRL	42
9.2.2.20 OP_SUB	42
9.2.2.21 OP_XOR	42
9.2.3 Function Documentation	42
9.2.3.1 alu_golden()	42
9.3 rtl/adder.v File Reference	43
9.3.1 Function Documentation	43
9.3.1.1 FAs_middle()	43
9.3.1.2 for()	43
9.3.1.3 if()	43
9.3.2 Variable Documentation	44

9.3.2.1 carry	44
9.3.2.2 i	44
9.3.2.3 internal_sum	44
9.3.2.4 sum	44
9.4 rtl/alu.v File Reference	44
9.4.1 Function Documentation	45
9.4.1.1 \$unsigned()	45
9.4.1.2 a()	45
9.4.1.3 b()	46
9.4.1.4 begin()	46
9.4.1.5 sum()	46
9.4.1.6 WIDTH()	46
9.4.2 Variable Documentation	46
9.4.2.1 ADD	46
9.4.2.2 adder_result	46
9.4.2.3 AND	46
9.4.2.4 AUIPC	46
9.4.2.5 BEQ	46
9.4.2.6 BGE	47
9.4.2.7 BGEU	47
9.4.2.8 BLT	47
9.4.2.9 BLTU	47
9.4.2.10 BNE	47
9.4.2.11 change_signal	47
9.4.2.12 cin	47
9.4.2.13 default	47
9.4.2.14 EBREAK	47
9.4.2.15 ECALL	47
9.4.2.16 FENCE	48
9.4.2.17 i_alu_ctrl_EX	48
9.4.2.18 i_rd1_EX	48
9.4.2.19 i_rd2_EX	48
9.4.2.20 last_bit_equal	48
9.4.2.21 LUI	48
9.4.2.22 not_i_rd2_EX	48
9.4.2.23 o_alu_result_EX	48
9.4.2.24 o_equal_EX	48
9.4.2.25 OR	48
9.4.2.26 rd2_operand	49
9.4.2.27 SLL	49
9.4.2.28 SLT	49
9.4.2.29 SLTU	49

9.4.2.30 SRA	49
9.4.2.31 SRL	49
9.4.2.32 SUB	49
9.4.2.33 XOR	49
9.5 rtl/alu_wrapper.sv File Reference	49
9.5.1 Detailed Description	50
9.5.2 Function Documentation	50
9.5.2.1 always_ff()	50
9.5.2.2 U_ALU()	50
9.5.3 Variable Documentation	50
9.5.3.1 i_rd1_EX	50
9.5.3.2 i_rd2_EX	50
9.5.3.3 o_alu_result_EX	50
9.5.3.4 o_equal_EX	51
9.6 rtl/full_adder.v File Reference	51
9.6.1 Variable Documentation	51
9.6.1.1 cout	51
9.6.1.2 sum	51
9.7 tb/s_interface.sv File Reference	51
9.7.1 Detailed Description	51
9.8 tb/tb_top.sv File Reference	51
9.8.1 Detailed Description	52
9.8.2 Function Documentation	52
9.8.2.1 run_test()	52
9.8.2.2 U_ALU_WRAPPER()	52
9.8.2.3 vif()	52
9.8.3 Variable Documentation	52
9.8.3.1 forever	52
9.8.3.2 rst	53
9.9 uvm/agent.sv File Reference	53
9.9.1 Detailed Description	53
9.10 agent.sv	53
9.11 uvm/alu_pkg.sv File Reference	54
9.11.1 Detailed Description	54
9.12 uvm/alu_sequence.sv File Reference	54
9.12.1 Detailed Description	55
9.13 alu_sequence.sv	55
9.14 uvm/coverage.sv File Reference	56
9.14.1 Detailed Description	56
9.15 coverage.sv	56
9.16 uvm/driver.sv File Reference	57
9.16.1 Detailed Description	57

9.17 driver.sv	58
9.18 uvm/env.sv File Reference	58
9.18.1 Detailed Description	59
9.19 env.sv	59
9.20 uvm/monitor.sv File Reference	59
9.20.1 Detailed Description	60
9.21 monitor.sv	60
9.22 uvm/pkt.sv File Reference	61
9.22.1 Detailed Description	61
9.23 pkt.sv	61
9.24 uvm/scoreboard.sv File Reference	62
9.24.1 Detailed Description	62
9.24.2 Function Documentation	62
9.24.2.1 alu_golden()	62
9.25 scoreboard.sv	63
9.26 uvm/sequencer.sv File Reference	64
9.26.1 Detailed Description	64
9.27 sequencer.sv	64
9.28 uvm/test.sv File Reference	65
9.28.1 Detailed Description	65
9.29 test.sv	65
Index	67

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

RTL design	11
Modules	11
Testbench components	11
Interfaces	12
UVM verification components	12

Chapter 2

Directory Hierarchy

2.1 Directories

filter	15
reference_model	15
reference_model.c	39
rtl	15
adder.v	43
alu.v	44
alu_wrapper.sv	49
full_adder.v	51
tb	16
s_interface.sv	51
tb_top.sv	51
uvm	16
agent.sv	53
alu_pkg.sv	54
alu_sequence.sv	54
coverage.sv	56
driver.sv	57
env.sv	58
monitor.sv	59
pkt.sv	61
scoreboard.sv	62
sequencer.sv	64
test.sv	65

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

s_interface	32
uvm_agent	
agent	17
uvm_driver	
driver	23
uvm_env	
env	25
uvm_monitor	
monitor	28
uvm_scoreboard	
scoreboard	33
uvm_sequence	
alu_sequence	19
uvm_sequence_item	
pkt	30
uvm_sequencer	
sequencer	35
uvm_subscriber	
coverage	21
uvm_test	
test	37

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

agent	UVM agent that connects the sequencer, driver, and monitor	17
alu_sequence	UVM sequence that drives ALU transactions until coverage reaches the target	19
coverage	UVM coverage subscriber for ALU transaction sampling	21
driver	UVM driver for ALU transactions	23
env	Top-level UVM environment for the ALU verification bench	25
monitor	UVM monitor for ALU transaction observation	28
pkt	Sequence item that models one ALU operation (stimulus + expected response fields)	30
s_interface	ALU stimulus/response interface shared by the DUT and the UVM environment	32
scoreboard	UVM scoreboard for ALU functional checking	33
sequencer	UVM sequencer for ALU transactions	35
test	Top-level UVM test for the ALU verification bench	37

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

reference_model/ reference_model.c	
Golden reference model for the ALU, called from the UVM scoreboard via DPI-C	39
rtl/ adder.v	43
rtl/ alu.v	44
rtl/ alu_wrapper.sv	
Registered wrapper around the combinational alu module, used as the DUT	49
rtl/ full_adder.v	51
tb/ s_interface.sv	
ALU stimulus/response interface shared by the DUT and the UVM environment	51
tb/ tb_top.sv	
Top-level testbench that instantiates the DUT, interface, and UVM test	51
uvm/ agent.sv	
UVM agent that bundles the sequencer, driver, and monitor	53
uvm/ alu_pkg.sv	
Package that includes the UVM classes used by the ALU verification environment	54
uvm/ alu_sequence.sv	
UVM sequence that drives ALU transactions until coverage reaches the target	54
uvm/ coverage.sv	
Coverage collector for ALU transactions	56
uvm/ driver.sv	
UVM driver that translates ALU transactions into interface signal activity	57
uvm/ env.sv	
UVM environment that instantiates the agent, scoreboard, and coverage collector	58
uvm/ monitor.sv	
UVM monitor that samples ALU transactions and publishes them to subscribers	59
uvm/ pkt.sv	
Transaction item used to model one ALU operation in the UVM environment	61
uvm/ scoreboard.sv	
Scoreboard that checks ALU transactions against the golden reference model	62
uvm/ sequencer.sv	
UVM sequencer that provides transactions to the driver	64
uvm/ test.sv	
Top-level UVM test that builds the environment and starts the main sequence	65

Chapter 6

Topic Documentation

6.1 RTL design

Register-transfer-level implementation of the ALU DUT.

Files

- file [alu_wrapper.sv](#)
Registered wrapper around the combinational [alu](#) module, used as the DUT.

6.1.1 Detailed Description

Register-transfer-level implementation of the ALU DUT.

6.2 Modules

6.3 Testbench components

Interface and assertion modules used by the verification environment.

Files

- file [s_interface.sv](#)
ALU stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

6.3.1 Detailed Description

Interface and assertion modules used by the verification environment.

6.4 Interfaces

Functions

- interface [s_interface](#) (input logic *clk*, input logic *reset*)

6.4.1 Detailed Description

6.4.2 Function Documentation

6.4.2.1 [s_interface\(\)](#)

```
interface s_interface (  
    input logic clk,  
    input logic reset)
```

< Operand rs1 ([i_rd1_EX](#)), driven by the driver.

< Operand rs2 ([i_rd2_EX](#)), driven by the driver.

< ALU opcode ([i_alu_ctrl_EX](#)), driven by the driver.

< ALU result ([o_alu_result_EX](#)), driven by the DUT.

< Comparison/branch-taken flag ([o_equal_EX](#)), driven by the DUT.

< Pulsed high by the driver while a transaction is valid.

Modport used by the UVM driver: drives the stimulus signals.

Modport used by the UVM monitor: observes every signal, drives none.

Modport used by the [alu_wrapper](#) DUT: consumes the stimulus, drives the result.

6.5 UVM verification components

UVM classes and utilities used to verify the ALU.

Files

- file [reference_model.c](#)
Golden reference model for the ALU, called from the UVM scoreboard via DPI-C.
- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [alu_sequence.sv](#)
UVM sequence that drives ALU transactions until coverage reaches the target.
- file [coverage.sv](#)
Coverage collector for ALU transactions.
- file [driver.sv](#)
UVM driver that translates ALU transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [monitor.sv](#)
UVM monitor that samples ALU transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one ALU operation in the UVM environment.
- file [scoreboard.sv](#)
Scoreboard that checks ALU transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

6.5.1 Detailed Description

UVM classes and utilities used to verify the ALU.

Chapter 7

Directory Documentation

7.1 filter Directory Reference

7.1.1 Detailed Description

7.1.2 SystemVerilog Doxygen filter

`idv_doxyfilter_sv.pl` converts SystemVerilog into C++-like code so Doxygen can parse it.

- **Author:** Sean O'Boyle, Intelligent Design Verification
- **Source:** <https://github.com/SeanOBoyle/DoxygenFilterSystemVerilog>
- **License:** GNU GPL v3 or later (see the header of the script). The file is included here unmodified.

It is wired up through `FILTER_PATTERNS` in `../Doxyfile`; run `doxygen` from the parent directory.

7.2 reference_model Directory Reference

Files

- file [reference_model.c](#)
Golden reference model for the ALU, called from the UVM scoreboard via DPI-C.

7.3 rtl Directory Reference

Files

- file [adder.v](#)
- file [alu.v](#)
- file [alu_wrapper.sv](#)
Registered wrapper around the combinational [alu](#) module, used as the DUT.
- file [full_adder.v](#)

7.4 tb Directory Reference

Files

- file [s_interface.sv](#)
ALU stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

7.5 uvm Directory Reference

Files

- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [alu_pkg.sv](#)
Package that includes the UVM classes used by the ALU verification environment.
- file [alu_sequence.sv](#)
UVM sequence that drives ALU transactions until coverage reaches the target.
- file [coverage.sv](#)
Coverage collector for ALU transactions.
- file [driver.sv](#)
UVM driver that translates ALU transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [monitor.sv](#)
UVM monitor that samples ALU transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one ALU operation in the UVM environment.
- file [scoreboard.sv](#)
Scoreboard that checks ALU transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

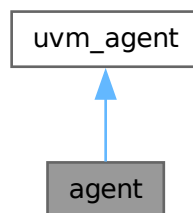
Chapter 8

Class Documentation

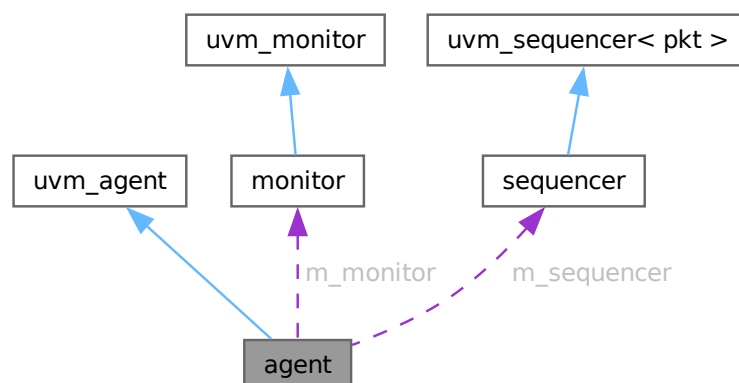
8.1 agent Class Reference

UVM agent that connects the sequencer, driver, and monitor.

Inheritance diagram for agent:



Collaboration diagram for agent:



Public Member Functions

- [uvm_component_utils](#) ([agent](#)) [driver](#) m_driver
Drives sequence items onto the DUT interface (null when passive).
- [new](#) (string name="agent", uvm_component parent=null)
- virtual void [build_phase](#) (uvm_phase phase)
Builds the sequencer/driver (active agent only) and the monitor.
- virtual void [connect_phase](#) (uvm_phase phase)
Connects the driver's seq_item port to the sequencer (active agent only).

Public Attributes

- [monitor](#) m_monitor
Samples DUT activity and publishes it (always created).
- [sequencer](#) m_sequencer
Arbitrates sequence items for the driver (null when passive).

8.1.1 Detailed Description

UVM agent that connects the sequencer, driver, and monitor.

Coordinates the active and passive verification components for the ALU testbench and exposes the common sequence and analysis interfaces. The sequencer and driver are only created when the agent is active; the monitor is always created.

8.1.2 Member Function Documentation

8.1.2.1 build_phase()

```
virtual void agent::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Builds the sequencer/driver (active agent only) and the monitor.

8.1.2.2 connect_phase()

```
virtual void agent::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the driver's seq_item port to the sequencer (active agent only).

8.1.2.3 new()

```
agent::new (  
    string name = "agent",  
    uvm_component parent = null) [inline]
```

8.1.2.4 uvm_component_utils()

```
agent::uvm_component_utils (
    agent )
```

Drives sequence items onto the DUT interface (null when passive).

8.1.3 Member Data Documentation

8.1.3.1 m_monitor

```
monitor agent::m_monitor
```

Samples DUT activity and publishes it (always created).

8.1.3.2 m_sequencer

```
sequencer agent::m_sequencer
```

Arbitrates sequence items for the driver (null when passive).

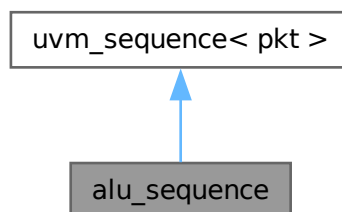
The documentation for this class was generated from the following file:

- [uvm/agent.sv](#)

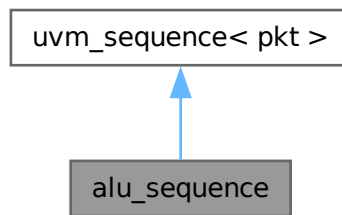
8.2 alu_sequence Class Reference

UVM sequence that drives ALU transactions until coverage reaches the target.

Inheritance diagram for alu_sequence:



Collaboration diagram for alu_sequence:



Public Member Functions

- `uvm_object_utils (alu_sequence)` real current_coverage=0
Latest coverage percentage reported by the coverage collector.
- `new` (string name="alu_sequence")
- virtual task `body` ()
Sends randomized ALU transactions until the coverage collector reports 100%.

Public Attributes

- int `num_packets` = 0
Number of transactions sent so far in this sequence.

8.2.1 Detailed Description

UVM sequence that drives ALU transactions until coverage reaches the target.

Repeatedly randomizes and sends a transaction, polling the coverage collector after each one, until 100% coverage is reached. Reads the coverage percentage from the configuration database key "cov_status".

8.2.2 Member Function Documentation

8.2.2.1 body()

```
virtual task alu_sequence::body () [inline], [virtual]
```

Sends randomized ALU transactions until the coverage collector reports 100%.

8.2.2.2 new()

```
alu_sequence::new (
    string name = "alu_sequence") [inline]
```

8.2.2.3 uvm_object_utils()

```
alu_sequence::uvm_object_utils (
    alu_sequence ) [pure virtual]
```

Latest coverage percentage reported by the coverage collector.

8.2.3 Member Data Documentation

8.2.3.1 num_packets

```
int alu_sequence::num_packets = 0
```

Number of transactions sent so far in this sequence.

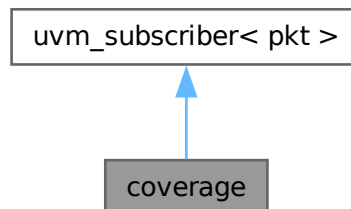
The documentation for this class was generated from the following file:

- [uvm/alu_sequence.sv](#)

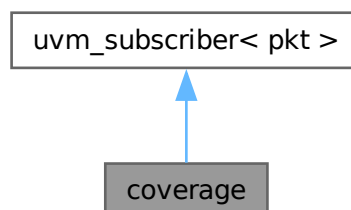
8.3 coverage Class Reference

UVM coverage subscriber for ALU transaction sampling.

Inheritance diagram for coverage:



Collaboration diagram for coverage:



Public Member Functions

- [uvm_component_utils](#) ([coverage](#)) [pkt](#) [tr](#)
Transaction currently being sampled by [cg_alu](#).
- covergroup [cg_alu](#) ()
Opcode x rs1-quadrant x rs2-quadrant cross coverage model.
- [new](#) (string name="coverage", [uvm_component](#) parent)
- virtual void [write](#) ([pkt](#) t)
Samples the covergroup for one transaction and republishes the running coverage percentage.

8.3.1 Detailed Description

UVM coverage subscriber for ALU transaction sampling.

Tracks opcode x operand-quadrant cross coverage and publishes the current percentage to the sequence layer through the configuration database under the key "cov_status".

8.3.2 Member Function Documentation

8.3.2.1 [cg_alu\(\)](#)

```
covergroup coverage::cg_alu () [inline]
```

Opcode x rs1-quadrant x rs2-quadrant cross coverage model.

One bin per ALU opcode (see [alu.v](#)'s localparams / [reference_model.c](#)'s OP_* defines).

Splits the full 32-bit rs1 range into 4 equal quadrants, so random stimulus hits every bin (a narrower range would make some bins statistically unreachable).

Same quadrant split as [cp_register_1](#), applied to rs2.

Full cross of opcode with both operand quadrants.

8.3.2.2 [new\(\)](#)

```
coverage::new (
    string name = "coverage",
    uvm_component parent) [inline]
```

8.3.2.3 [uvm_component_utils\(\)](#)

```
coverage::uvm_component_utils (
    coverage )
```

Transaction currently being sampled by [cg_alu](#).

8.3.2.4 write()

```
virtual void coverage::write (
    pkt t) [inline], [virtual]
```

Samples the covergroup for one transaction and republishes the running coverage percentage.

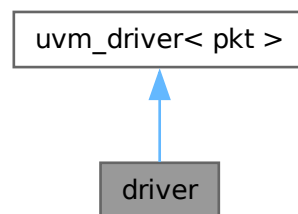
The documentation for this class was generated from the following file:

- [uvm/coverage.sv](#)

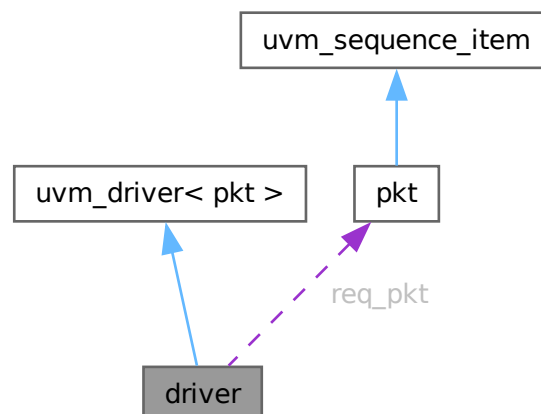
8.4 driver Class Reference

UVM driver for ALU transactions.

Inheritance diagram for driver:



Collaboration diagram for driver:



Public Member Functions

- `uvm_component_utils (driver)` virtual `s_interface.driver vif`
Driver modport of the DUT interface, obtained from the config DB (key "`vif`").
- `new` (string name="driver", uvm_component parent)
- virtual void `build_phase` (uvm_phase phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task `run_phase` (uvm_phase phase)
Waits for reset to be released, then drives sequence items forever.
- virtual task `drive_item` (pkt pkt_item)
Drives one ALU opcode/operand pair with `enable` pulsed for one clock cycle.

Public Attributes

- `pkt req_pkt`
Sequence item currently being driven.

8.4.1 Detailed Description

UVM driver for ALU transactions.

Converts each transaction item into the corresponding stimulus pulse on `s_interface`. Requires the virtual interface to be set in the configuration database under the key "`vif`".

8.4.2 Member Function Documentation

8.4.2.1 build_phase()

```
virtual void driver::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.4.2.2 drive_item()

```
virtual task driver::drive_item (
    pkt pkt_item) [inline], [virtual]
```

Drives one ALU opcode/operand pair with `enable` pulsed for one clock cycle.

8.4.2.3 new()

```
driver::new (
    string name = "driver",
    uvm_component parent) [inline]
```

8.4.2.4 run_phase()

```
virtual task driver::run_phase (
    uvm_phase phase)    [inline], [virtual]
```

Waits for reset to be released, then drives sequence items forever.

8.4.2.5 uvm_component_utils()

```
driver::uvm_component_utils (
    driver )
```

Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").

8.4.3 Member Data Documentation

8.4.3.1 req_pkt

```
pkt driver::req_pkt
```

Sequence item currently being driven.

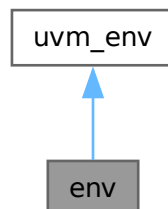
The documentation for this class was generated from the following file:

- [uvm/driver.sv](#)

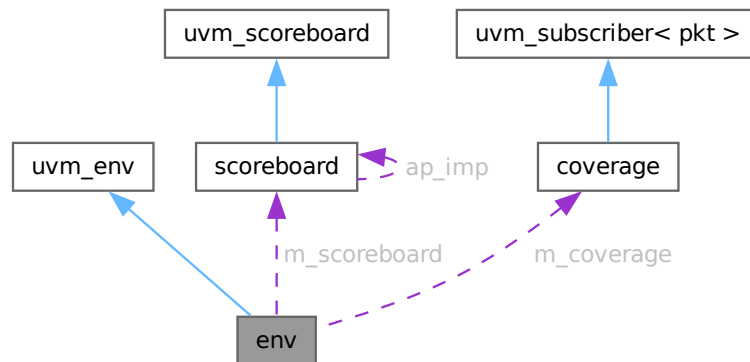
8.5 env Class Reference

Top-level UVM environment for the ALU verification bench.

Inheritance diagram for env:



Collaboration diagram for env:



Public Member Functions

- `uvm_component_utils (env) agent m_agent`
Agent that drives and monitors the DUT.
- `new (string name="env", uvm_component parent=null)`
- virtual void `build_phase (uvm_phase phase)`
Creates the agent, scoreboard, and coverage collector.
- virtual void `connect_phase (uvm_phase phase)`
Connects the monitor's analysis port to both the scoreboard and coverage collector.

Public Attributes

- `scoreboard m_scoreboard`
Checks monitored transactions against the reference model.
- `coverage m_coverage`
Collects functional coverage of monitored transactions.

8.5.1 Detailed Description

Top-level UVM environment for the ALU verification bench.

Instantiates the agent, scoreboard, and coverage collector and connects the monitored transactions to the checkers.

8.5.2 Member Function Documentation

8.5.2.1 build_phase()

```
virtual void env::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Creates the agent, scoreboard, and coverage collector.

8.5.2.2 connect_phase()

```
virtual void env::connect_phase (
    uvm_phase phase) [inline], [virtual]
```

Connects the monitor's analysis port to both the scoreboard and coverage collector.

8.5.2.3 new()

```
env::new (
    string name = "env",
    uvm_component parent = null) [inline]
```

8.5.2.4 uvm_component_utils()

```
env::uvm_component_utils (
    env )
```

Agent that drives and monitors the DUT.

8.5.3 Member Data Documentation

8.5.3.1 m_coverage

```
coverage env::m_coverage
```

Collects functional coverage of monitored transactions.

8.5.3.2 m_scoreboard

```
scoreboard env::m_scoreboard
```

Checks monitored transactions against the reference model.

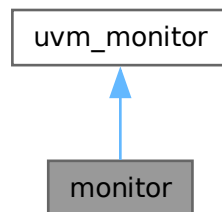
The documentation for this class was generated from the following file:

- [uvm/env.sv](#)

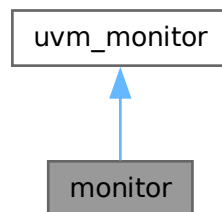
8.6 monitor Class Reference

UVM monitor for ALU transaction observation.

Inheritance diagram for monitor:



Collaboration diagram for monitor:



Public Member Functions

- `uvm_component_utils (monitor)` virtual s_interface.monitor `vif`
Monitor modport of the DUT interface, obtained from the config DB (key "`vif`").
- `new` (string name="monitor", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task `run_phase` (uvm_phase phase)
Samples one ALU transaction per iteration and publishes it.

Public Attributes

- `uvm_analysis_port< pkt > mon_analysis_port`
Publishes each observed transaction to the scoreboard and coverage collector.

8.6.1 Detailed Description

UVM monitor for ALU transaction observation.

Samples the DUT-visible signals on each transfer and publishes the resulting transaction object to downstream analysis components. Requires the virtual interface to be set in the configuration database under the key "[vif](#)".

8.6.2 Member Function Documentation

8.6.2.1 build_phase()

```
virtual void monitor::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.6.2.2 new()

```
monitor::new (  
    string name = "monitor",  
    uvm_component parent = null) [inline]
```

8.6.2.3 run_phase()

```
virtual task monitor::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Samples one ALU transaction per iteration and publishes it.

Captures the opcode/operands as soon as `enable` rises, then waits for `enable` to fall plus the [alu_wrapper](#) pipeline/settle delay before sampling the result/equal outputs, so the observed response always matches the just-captured stimulus.

8.6.2.4 uvm_component_utils()

```
monitor::uvm_component_utils (  
    monitor )
```

Monitor modport of the DUT interface, obtained from the config DB (key "[vif](#)").

8.6.3 Member Data Documentation

8.6.3.1 mon_analysis_port

```
uvm_analysis_port<pkt> monitor::mon_analysis_port
```

Publishes each observed transaction to the scoreboard and coverage collector.

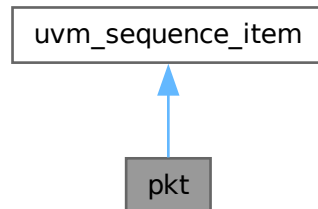
The documentation for this class was generated from the following file:

- [uvm/monitor.sv](#)

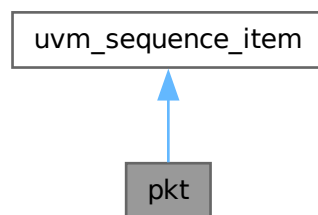
8.7 pkt Class Reference

Sequence item that models one ALU operation (stimulus + expected response fields).

Inheritance diagram for pkt:



Collaboration diagram for pkt:



Public Member Functions

- [uvm_object_utils_begin](#) (pkt) [uvm_field_int](#)(i_alu_ctrl_EX
 - UVM_DEFAULT [uvm_field_int](#) (i_rd1_EX, UVM_DEFAULT) [uvm_field_int](#)(i_rd2_EX
 - UVM_DEFAULT UVM_DEFAULT [uvm_field_int](#) (o_alu_result_EX, UVM_DEFAULT) [uvm_field_int](#)(o_equal_EX
 - UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT [uvm_object_utils_end](#) constraint [alu_ctrl_range](#) ()
- Restricts the randomized opcode to the 21 opcodes implemented by the ALU (see [alu.v](#)).*
- [new](#) (string name="pkt")

Public Attributes

- rand logic< 4:0 > [i_alu_ctrl_EX](#)
ALU opcode to drive.
- rand logic< 31:0 > [i_rd1_EX](#)
Operand rs1 to drive.

- rand logic< 31:0 > [i_rd2_EX](#)
Operand rs2 to drive.
- logic< 31:0 > [o_alu_result_EX](#)
ALU result, sampled by the monitor.
- logic [o_equal_EX](#)
Comparison/branch-taken flag, sampled by the monitor.

8.7.1 Detailed Description

Sequence item that models one ALU operation (stimulus + expected response fields).

Carries the opcode and operands randomized by the sequence/driver, and the result/equal fields filled in by the monitor for the scoreboard to check against the golden reference model.

8.7.2 Member Function Documentation

8.7.2.1 `alu_ctrl_range()`

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_object_utils_end constraint pkt::alu_ctrl_range ()
[inline]
```

Restricts the randomized opcode to the 21 opcodes implemented by the ALU (see [alu.v](#)).

8.7.2.2 `new()`

```
pkt::new (
    string name = "pkt") [inline]
```

8.7.2.3 `uvm_field_int()` [1/2]

```
UVM_DEFAULT pkt::uvm_field_int (
    i\_rdl\_EX ,
    UVM_DEFAULT )
```

8.7.2.4 `uvm_field_int()` [2/2]

```
UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    o\_alu\_result\_EX ,
    UVM_DEFAULT )
```

8.7.2.5 `uvm_object_utils_begin()`

```
pkt::uvm_object_utils_begin (
    pkt )
```

8.7.3 Member Data Documentation

8.7.3.1 i_alu_ctrl_EX

```
rand logic<4:0> pkt::i_alu_ctrl_EX
```

ALU opcode to drive.

8.7.3.2 i_rd1_EX

```
rand logic<31:0> pkt::i_rd1_EX
```

Operand rs1 to drive.

8.7.3.3 i_rd2_EX

```
rand logic<31:0> pkt::i_rd2_EX
```

Operand rs2 to drive.

8.7.3.4 o_alu_result_EX

```
logic<31:0> pkt::o_alu_result_EX
```

ALU result, sampled by the monitor.

8.7.3.5 o_equal_EX

```
logic pkt::o_equal_EX
```

Comparison/branch-taken flag, sampled by the monitor.

The documentation for this class was generated from the following file:

- [uvm/pkt.sv](#)

8.8 s_interface Interface Reference

ALU stimulus/response interface shared by the DUT and the UVM environment.

8.8.1 Detailed Description

ALU stimulus/response interface shared by the DUT and the UVM environment.

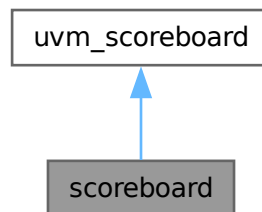
The documentation for this interface was generated from the following file:

- [tb/s_interface.sv](#)

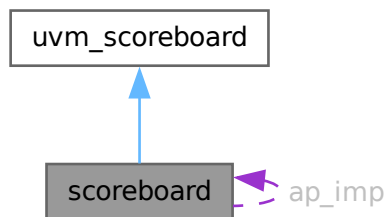
8.9 scoreboard Class Reference

UVM scoreboard for ALU functional checking.

Inheritance diagram for scoreboard:



Collaboration diagram for scoreboard:



Public Member Functions

- `uvm_component_utils (scoreboard) uvm_analysis_imp< pkt`
- `new` (string name="scoreboard", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Creates the analysis imp that receives transactions from the monitor.
- virtual void `write` (pkt data)
Analysis-port callback: checks one monitored transaction against `alu_golden`.
- virtual void `check_phase` (uvm_phase phase)
Fails the test if any mismatch was recorded by `write()`.
- virtual task `run_phase` (uvm_phase phase)
Run phase; unused, checking happens in `write()` and `check_phase()`.

Public Attributes

- [scoreboard ap_imp](#)
Receives transactions from the monitor's analysis port.
- `int num_errors = 0`
Number of mismatches found so far.

8.9.1 Detailed Description

UVM scoreboard for ALU functional checking.

Compares monitored ALU inputs/outputs with the DPI-C reference model and reports mismatches as test failures.

8.9.2 Member Function Documentation

8.9.2.1 build_phase()

```
virtual void scoreboard::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Creates the analysis imp that receives transactions from the monitor.

8.9.2.2 check_phase()

```
virtual void scoreboard::check_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fails the test if any mismatch was recorded by `write()`.

8.9.2.3 new()

```
scoreboard::new (  
    string name = "scoreboard",  
    uvm_component parent = null) [inline]
```

8.9.2.4 run_phase()

```
virtual task scoreboard::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Run phase; unused, checking happens in `write()` and `check_phase()`.

8.9.2.5 uvm_component_utils()

```
scoreboard::uvm_component_utils (  
    scoreboard )
```

8.9.2.6 write()

```
virtual void scoreboard::write (  
    pkt data) [inline], [virtual]
```

Analysis-port callback: checks one monitored transaction against `alu_golden`.

8.9.3 Member Data Documentation

8.9.3.1 ap_imp

```
scoreboard scoreboard::ap_imp
```

Receives transactions from the monitor's analysis port.

8.9.3.2 num_errors

```
int scoreboard::num_errors = 0
```

Number of mismatches found so far.

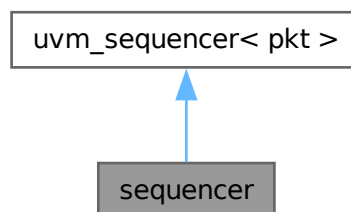
The documentation for this class was generated from the following file:

- [uvm/scoreboard.sv](#)

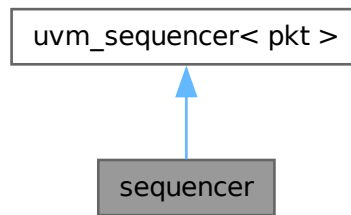
8.10 sequencer Class Reference

UVM sequencer for ALU transactions.

Inheritance diagram for sequencer:



Collaboration diagram for sequencer:



Public Member Functions

- [uvm_component_utils](#) ([sequencer](#)) new(string name

Public Attributes

- uvm_component [parent](#)

8.10.1 Detailed Description

UVM sequencer for ALU transactions.

Supplies randomized sequence items to the driver during the test run.

8.10.2 Member Function Documentation

8.10.2.1 [uvm_component_utils](#)()

```
sequencer::uvm_component_utils (
    sequencer ) [new]
```

8.10.3 Member Data Documentation

8.10.3.1 [parent](#)

```
uvm_component sequencer::parent
```

Initial value:

```
{
    super.new (name, parent)
}
```

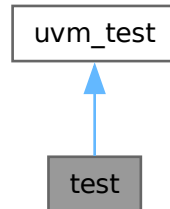
The documentation for this class was generated from the following file:

- [uvm/sequencer.sv](#)

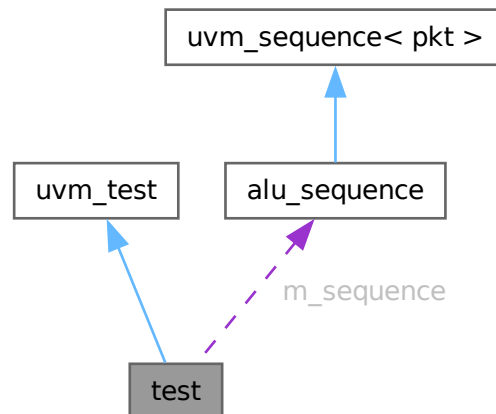
8.11 test Class Reference

Top-level UVM test for the ALU verification bench.

Inheritance diagram for test:



Collaboration diagram for test:



Public Member Functions

- `uvm_component_utils (test) env m_env`
Verification environment under test.
- `new` (string name="test", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Builds the verification environment.
- virtual void `end_of_elaboration_phase` (uvm_phase phase)
Prints the component topology once elaboration is complete.
- virtual task `run_phase` (uvm_phase phase)
Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

Public Attributes

- [alu_sequence m_sequence](#)
Main stimulus sequence, started on the agent's sequencer.

8.11.1 Detailed Description

Top-level UVM test for the ALU verification bench.

Builds the environment, starts the main sequence, and drives the full verification flow from the test root.

8.11.2 Member Function Documentation

8.11.2.1 build_phase()

```
virtual void test::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Builds the verification environment.

8.11.2.2 end_of_elaboration_phase()

```
virtual void test::end_of_elaboration_phase (
    uvm_phase phase) [inline], [virtual]
```

Prints the component topology once elaboration is complete.

8.11.2.3 new()

```
test::new (
    string name = "test",
    uvm_component parent = null) [inline]
```

8.11.2.4 run_phase()

```
virtual task test::run_phase (
    uvm_phase phase) [inline], [virtual]
```

Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

8.11.2.5 uvm_component_utils()

```
test::uvm_component_utils (
    test )
```

Verification environment under test.

8.11.3 Member Data Documentation

8.11.3.1 m_sequence

```
alu\_sequence test::m_sequence
```

Main stimulus sequence, started on the agent's sequencer.

The documentation for this class was generated from the following file:

- [uvm/test.sv](#)

Chapter 9

File Documentation

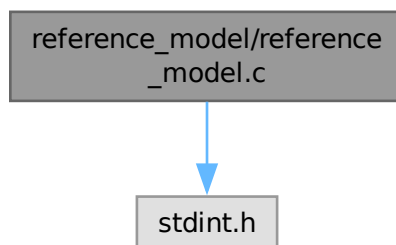
9.1 filter/README.md File Reference

9.2 reference_model/reference_model.c File Reference

Golden reference model for the ALU, called from the UVM scoreboard via DPI-C.

```
#include <stdint.h>
```

Include dependency graph for reference_model.c:



Macros

- `#define OP_AND 0`
- `#define OP_OR 1`
- `#define OP_XOR 2`
- `#define OP_ADD 3`
- `#define OP_SUB 4`
- `#define OP_SLL 5`
- `#define OP_SRL 6`
- `#define OP_SLT 7`
- `#define OP_SLTU 8`

- `#define OP_SRA 9`
- `#define OP_BEQ 10`
- `#define OP_BNE 11`
- `#define OP_BLT 12`
- `#define OP_BLTU 13`
- `#define OP_BGE 14`
- `#define OP_BGEU 15`
- `#define OP_LUI 16`
- `#define OP_AUIPC 17`
- `#define OP_FENCE 18`
- `#define OP_ECALL 19`
- `#define OP_EBREAK 20`

Functions

- void `alu_golden` (int `alu_ctrl`, unsigned int `rd1`, unsigned int `rd2`, unsigned int `*exp_result`, unsigned char `*exp_equal`)

Computes the expected ALU result and equal/comparison flag for one operation.

9.2.1 Detailed Description

Golden reference model for the ALU, called from the UVM scoreboard via DPI-C.

Implements the expected result and equal/comparison flag for every `alu_ctrl` opcode defined by the project, following the RISC-V ISA semantics (signed vs. unsigned comparisons, shift amount masking, etc.).

9.2.2 Macro Definition Documentation

9.2.2.1 OP_ADD

```
#define OP_ADD 3
```

9.2.2.2 OP_AND

```
#define OP_AND 0
```

9.2.2.3 OP_AUIPC

```
#define OP_AUIPC 17
```

9.2.2.4 OP_BEQ

```
#define OP_BEQ 10
```

9.2.2.5 OP_BGE

```
#define OP_BGE 14
```

9.2.2.6 OP_BGEU

```
#define OP_BGEU 15
```

9.2.2.7 OP_BLT

```
#define OP_BLT 12
```

9.2.2.8 OP_BLTU

```
#define OP_BLTU 13
```

9.2.2.9 OP_BNE

```
#define OP_BNE 11
```

9.2.2.10 OP_EBREAK

```
#define OP_EBREAK 20
```

9.2.2.11 OP_ECALL

```
#define OP_ECALL 19
```

9.2.2.12 OP_FENCE

```
#define OP_FENCE 18
```

9.2.2.13 OP_LUI

```
#define OP_LUI 16
```

9.2.2.14 OP_OR

```
#define OP_OR 1
```

9.2.2.15 OP_SLL

```
#define OP_SLL 5
```

9.2.2.16 OP_SLT

```
#define OP_SLT 7
```

9.2.2.17 OP_SLTU

```
#define OP_SLTU 8
```

9.2.2.18 OP_SRA

```
#define OP_SRA 9
```

9.2.2.19 OP_SRL

```
#define OP_SRL 6
```

9.2.2.20 OP_SUB

```
#define OP_SUB 4
```

9.2.2.21 OP_XOR

```
#define OP_XOR 2
```

9.2.3 Function Documentation

9.2.3.1 alu_golden()

```
void alu_golden (  
    int alu_ctrl,  
    unsigned int rd1,  
    unsigned int rd2,  
    unsigned int * exp_result,  
    unsigned char * exp_equal)
```

Computes the expected ALU result and equal/comparison flag for one operation.

Parameters

<i>alu_ctrl</i>	ALU opcode (see the OP_* defines above).
<i>rd1</i>	First operand (rs1), as an unsigned bit pattern.
<i>rd2</i>	Second operand (rs2), as an unsigned bit pattern.
<i>exp_result</i>	Output: expected <code>o_alu_result_EX</code> .
<i>exp_equal</i>	Output: expected <code>o_equal_EX</code> (comparison/branch-taken flag).

9.3 rtl/adder.v File Reference

Functions

- generate `for (i=0;i< WIDTH;i=i+1) begin if(i`
- end else `if (i==WIDTH - 1) begin assign internal_sum[i]`
- end else `begin full_adder FAs_middle (.a(a[i]),.b(b[i]),.cin(carry[i-1]),.sum(internal_sum[i]),.cout(carry[i]))`

Variables

- wire[WIDTH-1:0] `internal_sum`
- wire[WIDTH-1:0] `carry`
- genvar `i`
- assign `sum = internal_sum`

9.3.1 Function Documentation

9.3.1.1 FAs_middle()

```
end else begin full_adder FAs_middle (
    . aa[i],
    . bb[i],
    . cincarry[i-1],
    . suminternal_sum[i],
    . coutcarry[i])
```

9.3.1.2 for()

```
generate for ()
```

9.3.1.3 if()

```
end else if (
    i  = =WIDTH - 1)
```

9.3.2 Variable Documentation

9.3.2.1 carry

```
wire [WIDTH-1:0] carry
```

9.3.2.2 i

```
genvar i
```

9.3.2.3 internal_sum

```
wire [WIDTH-1:0] internal_sum
```

9.3.2.4 sum

```
assign sum = internal_sum
```

9.4 rtl/alu.v File Reference

Functions

- `adder WIDTH (WIDTH)) U_ADDER(.cin(cin)`
- `adder a (i_rd1_EX)`
- `adder b (rd2_operand)`
- `adder sum (adder_result)`
- `always * begin (i_alu_ctrl_EX==SUB) begin rd2_operand=not i_rd2_EX;cin=1 'b1;end else begin cin=1 'b0;rd2_operand=i_rd2_EX;end end :invert_register_2 always @(*) begin :operations case(i_alu_ctrl_EX) AND:begin o_alu_result_EX=i_rd1_EX & i_rd2_EX;o_equal_EX=1 'b0;end OR:begin o_alu_result_EX=i_rd1_EX|i_rd2_EX;o_equal_EX=1 'b0;end XOR:begin o_alu_result_EX=i_rd1_EX ^ i_rd2_EX;o_equal_EX=1 'b0;end ADD:begin o_alu_result_EX=adder_result;o_equal_EX=1 'b0;end SUB:begin o_alu_result_EX=adder_result;o_equal_EX=1 'b0;end SLL:begin o_alu_result_EX=i_rd1_EX<<i_rd2_EX[4:0];o_equal_EX=1 'b0;end SRL:begin o_alu_result_EX=i_rd1_EX >> i_rd2_EX[4:0];o_equal_EX=1 'b0;end SLT:begin o_alu_result_EX={{WIDTH - 1{1 'b0}},($signed(i_rd1_EX)<$signed(i_rd2_EX))};o_equal_EX=1 'b0;end end $unsigned (i_rd1_EX)<$unsigned(i_rd2_EX))}`

Variables

- input wire[4:0] `i_alu_ctrl_EX`
- input wire[WIDTH-1:0] `i_rd1_EX`
- input wire[WIDTH-1:0] `i_rd2_EX`
- output reg[WIDTH-1:0] `o_alu_result_EX`
- output reg `o_equal_EX` = (`i_rd1_EX` == `i_rd2_EX`)
- localparam logic[4:0] `AND` = 5'b00000
- localparam logic[4:0] `OR` = 5'b00001
- localparam logic[4:0] `XOR` = 5'b00010
- localparam logic[4:0] `ADD` = 5'b00011
- localparam logic[4:0] `SUB` = 5'b00100
- localparam logic[4:0] `SLL` = 5'b00101
- localparam logic[4:0] `SRL` = 5'b00110
- localparam logic[4:0] `SLT` = 5'b00111
- localparam logic[4:0] `SLTU` = 5'b01000
- localparam logic[4:0] `SRA` = 5'b01001
- localparam logic[4:0] `BEQ` = 5'b01010
- localparam logic[4:0] `BNE` = 5'b01011
- localparam logic[4:0] `BLT` = 5'b01100
- localparam logic[4:0] `BLTU` = 5'b01101
- localparam logic[4:0] `BGE` = 5'b01110
- localparam logic[4:0] `BGEU` = 5'b01111
- localparam logic[4:0] `LUI` = 5'b10000
- localparam logic[4:0] `AUIPC` = 5'b10001
- localparam logic[4:0] `FENCE` = 5'b10010
- localparam logic[4:0] `ECALL` = 5'b10011
- localparam logic[4:0] `EBREAK` = 5'b10100
- wire[WIDTH-1:0] `not_i_rd2_EX` = ~`i_rd2_EX`
- reg[WIDTH-1:0] `rd2_operand`
- reg[WIDTH-1:0] `adder_result`
- reg `cin`
- reg `change_signal`
- reg `last_bit_equal`
- end default

9.4.1 Function Documentation

9.4.1.1 \$unsigned()

```
end $unsigned (
    i_rd1_EX )
```

9.4.1.2 a()

```
adder a (
    i_rd1_EX )
```

9.4.1.3 b()

```
adder b (
    rd2_operand )
```

9.4.1.4 begin()

```
always * begin (
    i_alu_ctrl_EX == SUB) &
```

9.4.1.5 sum()

```
adder sum (
    adder_result )
```

9.4.1.6 WIDTH()

```
adder WIDTH (
    WIDTH )
```

9.4.2 Variable Documentation

9.4.2.1 ADD

```
localparam logic [4:0] ADD = 5'b00011
```

9.4.2.2 adder_result

```
reg [WIDTH-1:0] adder_result
```

9.4.2.3 AND

```
localparam logic [4:0] AND = 5'b00000
```

9.4.2.4 AUIPC

```
localparam logic [4:0] AUIPC = 5'b10001
```

9.4.2.5 BEQ

```
end BEQ = 5'b01010
```

9.4.2.6 BGE

```
end BGE = 5'b01110
```

9.4.2.7 BGEU

```
end BGEU = 5'b01111
```

9.4.2.8 BLT

```
end BLT = 5'b01100
```

9.4.2.9 BLTU

```
end BLTU = 5'b01101
```

9.4.2.10 BNE

```
end BNE = 5'b01011
```

9.4.2.11 change_signal

```
reg change_signal
```

9.4.2.12 cin

```
reg cin
```

9.4.2.13 default

```
end default
```

9.4.2.14 EBREAK

```
localparam logic [4:0] EBREAK = 5'b10100
```

9.4.2.15 ECALL

```
localparam logic [4:0] ECALL = 5'b10011
```

9.4.2.16 FENCE

```
localparam logic [4:0] FENCE = 5'b10010
```

9.4.2.17 i_alu_ctrl_EX

```
input wire [4:0] i_alu_ctrl_EX
```

9.4.2.18 i_rd1_EX

```
input wire [WIDTH-1:0] i_rd1_EX
```

9.4.2.19 i_rd2_EX

```
input wire [WIDTH-1:0] i_rd2_EX
```

9.4.2.20 last_bit_equal

```
reg last_bit_equal
```

9.4.2.21 LUI

```
end LUI = 5'b10000
```

9.4.2.22 not_i_rd2_EX

```
assign not_i_rd2_EX = ~i_rd2_EX
```

9.4.2.23 o_alu_result_EX

```
output reg [WIDTH-1:0] o_alu_result_EX
```

9.4.2.24 o_equal_EX

```
o_equal_EX = (i_rd1_EX == i_rd2_EX)
```

9.4.2.25 OR

```
localparam logic [4:0] OR = 5'b00001
```

9.4.2.26 rd2_operand

```
reg [WIDTH-1:0] rd2_operand
```

9.4.2.27 SLL

```
localparam logic [4:0] SLL = 5'b00101
```

9.4.2.28 SLT

```
localparam logic [4:0] SLT = 5'b00111
```

9.4.2.29 SLTU

```
end SLTU = 5'b01000
```

9.4.2.30 SRA

```
end SRA = 5'b01001
```

9.4.2.31 SRL

```
localparam logic [4:0] SRL = 5'b00110
```

9.4.2.32 SUB

```
localparam logic [4:0] SUB = 5'b00100
```

9.4.2.33 XOR

```
localparam logic [4:0] XOR = 5'b00010
```

9.5 rtl/alu_wrapper.sv File Reference

Registered wrapper around the combinational [alu](#) module, used as the DUT.

Functions

- [alu](#)<.WIDTH(32)> [U_ALU](#) (.i_alu_ctrl_EX(i_alu_ctrl_EX),.i_rd1_EX(i_rd1_EX),.i_rd2_EX(i_rd2_EX),.o_alu_result_EX(o_alu_r
- [always_ff](#) (posedge vif.clk or negedge vif.reset)

Variables

- logic< 31:0 > [i_rd1_EX](#)
- logic< 31:0 > [i_rd2_EX](#)
- logic< 31:0 > [o_alu_result_EX](#)
- logic [o_equal_EX](#)

9.5.1 Detailed Description

Registered wrapper around the combinational [alu](#) module, used as the DUT.

Latches [i_alu_ctrl_EX/i_rd1_EX/i_rd2_EX](#) from the `s_interface.dut` modport on the rising edge of `clk`, feeds them into the combinational [alu](#) instance, and registers the result back onto `vif.alu_result`/`vif.equal` a few time units later. Held at zero while `vif.reset` is deasserted (active-low).

9.5.2 Function Documentation

9.5.2.1 `always_ff()`

```
always_ff (
    posedge vif.clk or negedge vif.  reset)
```

9.5.2.2 `U_ALU()`

```
alu<.WIDTH(32)> U_ALU (
    .  i_alu_ctrl_EXi_alu_ctrl_EX,
    .  i_rd1_EXi_rd1_EX,
    .  i_rd2_EXi_rd2_EX,
    .  o_alu_result_EXo_alu_result_EX,
    .  o_equal_EXo_equal_EX)
```

9.5.3 Variable Documentation

9.5.3.1 `i_rd1_EX`

```
logic<31:0> i_rd1_EX
```

9.5.3.2 `i_rd2_EX`

```
logic<31:0> i_rd2_EX
```

9.5.3.3 `o_alu_result_EX`

```
logic<31:0> o_alu_result_EX
```

9.5.3.4 o_equal_EX

```
logic o_equal_EX
```

9.6 rtl/full_adder.v File Reference

Variables

- assign `sum` = `a` ^ `b` ^ `cin`
- assign `cout` = (`a` & `b`) | (`b` & `cin`) | (`a` & `cin`)

9.6.1 Variable Documentation

9.6.1.1 cout

```
assign cout = (a & b) | (b & cin) | (a & cin)
```

9.6.1.2 sum

```
assign sum = a ^ b ^ cin
```

9.7 tb/s_interface.sv File Reference

ALU stimulus/response interface shared by the DUT and the UVM environment.

Functions

- interface `s_interface` (input logic `clk`, input logic `reset`)

9.7.1 Detailed Description

ALU stimulus/response interface shared by the DUT and the UVM environment.

Declares the ALU operand/opcode/result/handshake signals and exposes the `driver`, `monitor`, and `dut` mod-ports used by the UVM driver, the UVM monitor, and the `alu_wrapper` DUT respectively.

9.8 tb/tb_top.sv File Reference

Top-level testbench that instantiates the DUT, interface, and UVM test.

Functions

- `s_interface vif (.clk(clk),.reset(rst))`
- `alu_wrapper U_ALU_WRAPPER (.vif(vif))`
- `run_test ("test")`

Variables

- bit `rst`
- `forever`

9.8.1 Detailed Description

Top-level testbench that instantiates the DUT, interface, and UVM test.

Connects `s_interface`, the `alu_wrapper` DUT, and the UVM test entry point in a single simulation environment.

9.8.2 Function Documentation

9.8.2.1 run_test()

```
run_test (
    "test" )
```

9.8.2.2 U_ALU_WRAPPER()

```
alu_wrapper U_ALU_WRAPPER (
    . vifvif)
```

9.8.2.3 vif()

```
s_interface vif (
    . clkclk,
    . resetrst)
```

9.8.3 Variable Documentation

9.8.3.1 forever

```
forever
```

Initial value:

```
{
    rst = 0
```

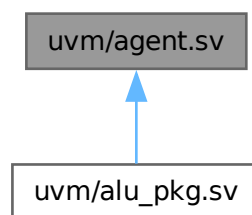
9.8.3.2 rst

```
bit rst
```

9.9 uvm/agent.sv File Reference

UVM agent that bundles the sequencer, driver, and monitor.

This graph shows which files directly or indirectly include this file:



Classes

- class [agent](#)
UVM agent that connects the sequencer, driver, and monitor.

9.9.1 Detailed Description

UVM agent that bundles the sequencer, driver, and monitor.

Implements the standard UVM agent structure for the ALU verification environment, creating active and passive components as needed.

9.10 agent.sv

[Go to the documentation of this file.](#)

```

00001
00018 class agent extends uvm_agent;
00019     `uvm_component_utils (agent)
00020
00021     driver m_driver;
00022
00023     monitor m_monitor;
00024
00025     sequencer m_sequencer;
00026
00027     function new (string name = "agent", uvm_component parent = null);
00028         super.new (name, parent);
00029     endfunction
  
```

```

00030
00032     virtual function void build_phase (uvm_phase phase);
00033         super.build_phase (phase);
00034
00035         if (get_is_active()) begin
00036             m_sequencer = sequencer::type_id::create ("sequencer", this);
00037             m_driver = driver::type_id::create ("driver", this);
00038         end
00039
00040         m_monitor = monitor::type_id::create ("monitor", this);
00041
00042     endfunction
00043
00044     virtual function void connect_phase (uvm_phase phase);
00045         super.connect_phase (phase);
00046         if (get_is_active()) begin
00047             m_driver.seq_item_port.connect (m_sequencer.seq_item_export);
00048         end
00049     endfunction
00050
00051
00052 endclass

```

9.11 uvm/alu_pkg.sv File Reference

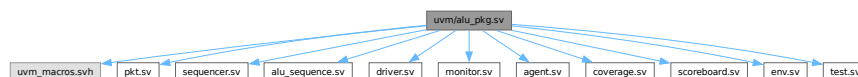
Package that includes the UVM classes used by the ALU verification environment.

```

#include "uvm_macros.svh"
#include "pkt.sv"
#include "sequencer.sv"
#include "alu_sequence.sv"
#include "driver.sv"
#include "monitor.sv"
#include "agent.sv"
#include "coverage.sv"
#include "scoreboard.sv"
#include "env.sv"
#include "test.sv"

```

Include dependency graph for alu_pkg.sv:



9.11.1 Detailed Description

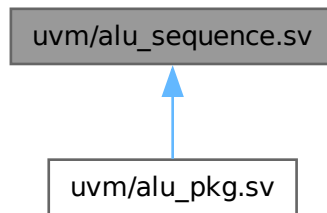
Package that includes the UVM classes used by the ALU verification environment.

Centralizes the inclusion of all verification components required to build the ALU UVM testbench.

9.12 uvm/alu_sequence.sv File Reference

UVM sequence that drives ALU transactions until coverage reaches the target.

This graph shows which files directly or indirectly include this file:



Classes

- class [alu_sequence](#)

UVM sequence that drives ALU transactions until coverage reaches the target.

9.12.1 Detailed Description

UVM sequence that drives ALU transactions until coverage reaches the target.

Repeatedly sends randomized transactions while waiting for the coverage collector to report completion of the sampling process.

9.13 alu_sequence.sv

[Go to the documentation of this file.](#)

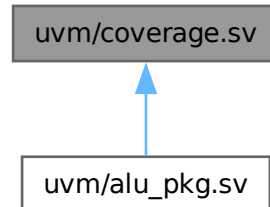
```

00001
00017 class alu_sequence extends uvm_sequence #(pkt);
00018     `uvm_object_utils(alu_sequence)
00019
00020     real current_coverage = 0;
00021
00022     int num_packets = 0;
00023
00024     function new (string name = "alu_sequence");
00025         super.new(name);
00026     endfunction
00027
00029     virtual task body();
00030         pkt packet;
00031         `uvm_create(packet)
00032         while (current_coverage < 100.0) begin
00033             `uvm_rand_send(packet)
00034             num_packets++;
00035             void' (uvm_config_db#(real)::get(null, "*", "cov_status", current_coverage));
00036             `uvm_info("SEQ", $sformatf("Status: %0.2f%%", current_coverage), UVM_LOW)
00037         end
00038         `uvm_info("SEQ", $sformatf("Total packets sent: %0d", num_packets), UVM_LOW)
00039     endtask
00040
00041 endclass
  
```

9.14 uvm/coverage.sv File Reference

Coverage collector for ALU transactions.

This graph shows which files directly or indirectly include this file:



Classes

- class [coverage](#)
UVM coverage subscriber for ALU transaction sampling.

9.14.1 Detailed Description

Coverage collector for ALU transactions.

Samples observed ALU transactions and updates the coverage model used to determine whether the verification goal has been reached.

9.15 coverage.sv

[Go to the documentation of this file.](#)

```

00001
00017 class coverage extends uvm_subscriber #(pkt);
00018   `uvm_component_utils(coverage)
00019
00020   pkt tr;
00021
00023   covergroup cg_alu;
00024     option.per_instance = 1;
00025
00027     cp_control: coverpoint tr.i_alu_ctrl_EX {
00028       bins controls[21] = {[0:20]};
00029     }
00030
00033     cp_register_1: coverpoint tr.i_rd1_EX {
00034       bins low      = {[32'h0000_0000:32'h3FFF_FFFF]};
00035       bins low_med  = {[32'h4000_0000:32'h7FFF_FFFF]};
00036       bins high_med = {[32'h8000_0000:32'hBFFF_FFFF]};
00037       bins high     = {[32'hC000_0000:32'hFFFF_FFFF]};
00038     }
00039
00041     cp_register_2: coverpoint tr.i_rd2_EX {
00042       bins low      = {[32'h0000_0000:32'h3FFF_FFFF]};
00043       bins low_med  = {[32'h4000_0000:32'h7FFF_FFFF]};

```

```

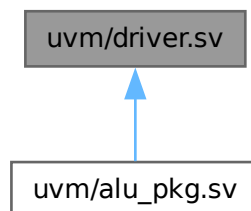
00044     bins high_med = {[32'h8000_0000:32'hBFFF_FFFF]};
00045     bins high      = {[32'hC000_0000:32'hFFFF_FFFF]};
00046   }
00047
00049   cp_cross_operations: cross cp_control, cp_register_1, cp_register_2;
00050
00051 endgroup
00052
00053 function new(string name = "coverage", uvm_component parent);
00054   super.new(name, parent);
00055   cg_alu = new();
00056   cg_alu.set_inst_name("alu_cov");
00057 endfunction
00058
00060 virtual function void write(pkt t);
00061   this.tr = t;
00062   cg_alu.sample();
00063   uvm_config_db#(real)::set(null, "*", "cov_status", cg_alu.get_inst_coverage());
00064 endfunction
00065
00066 endclass

```

9.16 uvm/driver.sv File Reference

UVM driver that translates ALU transactions into interface signal activity.

This graph shows which files directly or indirectly include this file:



Classes

- class `driver`
UVM driver for ALU transactions.

9.16.1 Detailed Description

UVM driver that translates ALU transactions into interface signal activity.

Receives sequence items from the sequencer and drives the ALU opcode and operands onto `s_interface` for one clock cycle each.

9.17 driver.sv

[Go to the documentation of this file.](#)

```

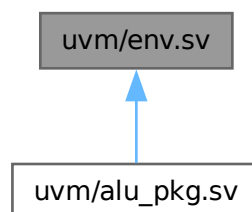
00001
00017 class driver extends uvm_driver #(pkt);
00018     `uvm_component_utils (driver)
00019
00020     virtual s_interface.driver vif;
00021
00022     pkt req_pkt;
00023
00024     function new (string name = "driver", uvm_component parent);
00025         super.new(name, parent);
00026     endfunction
00027
00029     virtual function void build_phase (uvm_phase phase);
00030         super.build_phase(phase);
00031         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00032             `uvm_fatal (get_type_name (), "Didn't get handle to virtual interface if_name")
00033         end
00034     endfunction
00035
00037     virtual task run_phase(uvm_phase phase);
00038         super.run_phase(phase);
00039
00040         wait (vif.reset == 1'b1);
00041
00042         forever begin
00043             seq_item_port.get_next_item(req_pkt);
00044             drive_item(req_pkt);
00045             seq_item_port.item_done();
00046         end
00047     endtask
00048
00050     virtual task drive_item (pkt pkt_item);
00051         @(posedge vif.clk);
00052         vif.register_1 <= pkt_item.i_rd1_EX;
00053         vif.register_2 <= pkt_item.i_rd2_EX;
00054         vif.alu_ctrl <= pkt_item.i_alu_ctrl_EX;
00055         vif.enable <= '1;
00056
00057         @(posedge vif.clk);
00058         vif.enable <= '0;
00059     endtask
00060
00061
00062 endclass

```

9.18 uvm/env.sv File Reference

UVM environment that instantiates the agent, scoreboard, and coverage collector.

This graph shows which files directly or indirectly include this file:



Classes

- class `env`

Top-level UVM environment for the ALU verification bench.

9.18.1 Detailed Description

UVM environment that instantiates the agent, scoreboard, and coverage collector.

Connects the main verification components so that transactions generated by the agent can be checked and tracked by the scoreboard and coverage.

9.19 env.sv

[Go to the documentation of this file.](#)

```

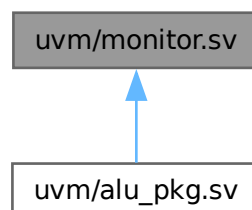
00001
00016 class env extends uvm_env;
00017     `uvm_component_utils (env)
00018
00019     agent m_agent;
00020
00021     scoreboard m_scoreboard;
00022
00023     coverage m_coverage;
00024
00025     function new (string name = "env", uvm_component parent = null);
00026         super.new (name, parent);
00027     endfunction
00028
00030     virtual function void build_phase (uvm_phase phase);
00031         super.build_phase (phase);
00032         m_agent = agent::type_id::create ("agent", this);
00033         m_scoreboard = scoreboard::type_id::create ("scoreboard", this);
00034         m_coverage = coverage::type_id::create ("coverage", this);
00035     endfunction
00036
00038     virtual function void connect_phase (uvm_phase phase);
00039         super.connect_phase (phase);
00040         m_agent.m_monitor.mon_analysis_port.connect(m_scoreboard.ap_imp);
00041         m_agent.m_monitor.mon_analysis_port.connect(m_coverage.analysis_export);
00042     endfunction
00043
00044 endclass

```

9.20 uvm/monitor.sv File Reference

UVM monitor that samples ALU transactions and publishes them to subscribers.

This graph shows which files directly or indirectly include this file:



Classes

- class [monitor](#)

UVM monitor for ALU transaction observation.

9.20.1 Detailed Description

UVM monitor that samples ALU transactions and publishes them to subscribers.

Observes [s_interface](#) for each transaction and forwards the completed opcode/operands/result/equal fields to the scoreboard and coverage collector.

9.21 monitor.sv

[Go to the documentation of this file.](#)

```

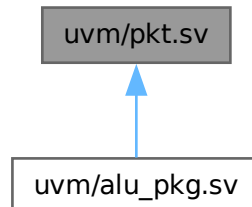
00001
00018 class monitor extends uvm_monitor;
00019     `uvm_component_utils(monitor)
00020
00021     virtual s_interface.monitor vif;
00022
00023     uvm_analysis_port #(pkt) mon_analysis_port;
00024
00025     function new(string name = "monitor", uvm_component parent = null);
00026         super.new(name, parent);
00027     endfunction
00028
00030     virtual function void build_phase(uvm_phase phase);
00031         super.build_phase(phase);
00032         mon_analysis_port = new("mon_analysis_port", this);
00033
00034         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00035             `uvm_error(get_type_name(), "DUT interface not found")
00036         end
00037     endfunction
00038
00047     virtual task run_phase(uvm_phase phase);
00048         super.run_phase(phase);
00049
00050         forever begin
00051             pkt mon_pkt;
00052             mon_pkt = pkt::type_id::create("mon_pkt", this);
00053
00054             @(posedge vif.enable);
00055
00056             mon_pkt.i_alu_ctrl_EX = vif.alu_ctrl;
00057             mon_pkt.i_rd1_EX = vif.register_1;
00058             mon_pkt.i_rd2_EX = vif.register_2;
00059
00060             // alu_wrapper registers its inputs and drives the outputs #5 after
00061             // the clock edge that deasserts enable, so wait past both before sampling.
00062             @(negedge vif.enable);
00063             #6;
00064
00065             mon_pkt.o_alu_result_EX = vif.alu_result;
00066             mon_pkt.o_equal_EX = vif.equal;
00067
00068             `uvm_info(get_type_name(),
00069                 $sformatf("Monitored CTRL=%0d, RD1=%0h, RD2=%0h, RESULT=%0h, EQUAL=%0b",
00070                     mon_pkt.i_alu_ctrl_EX, mon_pkt.i_rd1_EX, mon_pkt.i_rd2_EX,
00071                     mon_pkt.o_alu_result_EX, mon_pkt.o_equal_EX),
00072                 UVM_LOW)
00073
00074             mon_analysis_port.write(mon_pkt);
00075         end
00076     endtask
00077
00078 endclass

```

9.22 uvm/pkt.sv File Reference

Transaction item used to model one ALU operation in the UVM environment.

This graph shows which files directly or indirectly include this file:



Classes

- class `pkt`

Sequence item that models one ALU operation (stimulus + expected response fields).

9.22.1 Detailed Description

Transaction item used to model one ALU operation in the UVM environment.

Encapsulates the opcode and operands driven into the ALU, plus the result and equal flag sampled back from it.

9.23 pkt.sv

[Go to the documentation of this file.](#)

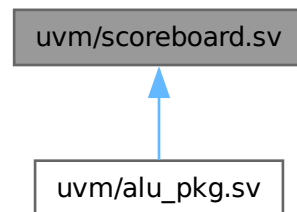
```

00001
00017 class pkt extends uvm_sequence_item;
00018
00019     rand logic [4:0] i_alu_ctrl_EX;
00020     rand logic [31:0] i_rd1_EX;
00021     rand logic [31:0] i_rd2_EX;
00022     logic [31:0] o_alu_result_EX;
00023     logic o_equal_EX;
00024
00025     `uvm_object_utils_begin(pkt)
00026         `uvm_field_int (i_alu_ctrl_EX, UVM_DEFAULT)
00027         `uvm_field_int (i_rd1_EX, UVM_DEFAULT)
00028         `uvm_field_int (i_rd2_EX, UVM_DEFAULT)
00029         `uvm_field_int (o_alu_result_EX, UVM_DEFAULT)
00030         `uvm_field_int (o_equal_EX, UVM_DEFAULT)
00031     `uvm_object_utils_end
00032
00033     constraint alu_ctrl_range {
00034         i_alu_ctrl_EX inside {[0:20]};
00035     }
00036
00037     function new(string name = "pkt");
00038         super.new(name);
00039     endfunction
00040
00041
00042 endclass
  
```

9.24 uvm/scoreboard.sv File Reference

Scoreboard that checks ALU transactions against the golden reference model.

This graph shows which files directly or indirectly include this file:



Classes

- class [scoreboard](#)
UVM scoreboard for ALU functional checking.

Functions

- void [alu_golden](#) (input int alu_ctrl, input int unsigned rd1, input int unsigned rd2, output int unsigned exp_↔ result, output byte unsigned exp_equal)
DPI-C import of the golden reference model (see [reference_model.c](#)).

9.24.1 Detailed Description

Scoreboard that checks ALU transactions against the golden reference model.

Receives monitored transactions and validates them against the DPI-C reference model implemented in the C source file.

9.24.2 Function Documentation

9.24.2.1 alu_golden()

```

void alu_golden (
    input int alu_ctrl,
    input int unsigned rd1,
    input int unsigned rd2,
    output int unsigned exp_result,
    output byte unsigned exp_equal)
  
```

DPI-C import of the golden reference model (see [reference_model.c](#)).

Parameters

<i>alu_ctrl</i>	ALU opcode.
<i>rd1</i>	Operand rs1.
<i>rd2</i>	Operand rs2.
<i>exp_result</i>	Expected o_alu_result_EX .
<i>exp_equal</i>	Expected o_equal_EX (only the low bit is meaningful).

9.25 scoreboard.sv

[Go to the documentation of this file.](#)

```

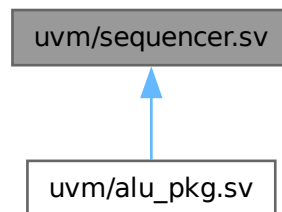
00001
00018 import "DPI-C" function void alu_golden(
00019     input  int      alu_ctrl,
00020     input  int unsigned rd1,
00021     input  int unsigned rd2,
00022     output int unsigned exp_result,
00023     output byte unsigned exp_equal
00024 );
00025
00033 class scoreboard extends uvm_scoreboard;
00034     `uvm_component_utils(scoreboard)
00035
00036     uvm_analysis_imp #(pkt, scoreboard) ap_imp;
00037
00038     int num_errors = 0;
00039
00040     function new(string name = "scoreboard", uvm_component parent = null);
00041         super.new(name, parent);
00042     endfunction
00043
00045     virtual function void build_phase(uvm_phase phase);
00046         super.build_phase(phase);
00047         ap_imp = new("ap_imp", this);
00048     endfunction
00049
00051     virtual function void write(pkt data);
00052         logic [31:0] exp_result;
00053         byte unsigned exp_equal_raw;
00054         logic exp_equal;
00055
00056         alu_golden(data.i_alu_ctrl_EX, data.i_rd1_EX, data.i_rd2_EX, exp_result, exp_equal_raw);
00057         exp_equal = exp_equal_raw[0];
00058
00059         if (data.o_alu_result_EX == exp_result && data.o_equal_EX == exp_equal) begin
00060             `uvm_info("SCOREBOARD",
00061                 $sformatf("PASS: ctrl=%0d rd1=%0h rd2=%0h result=%0h equal=%0b",
00062                     data.i_alu_ctrl_EX, data.i_rd1_EX, data.i_rd2_EX, data.o_alu_result_EX, data.o_equal_EX),
00063                     UVM_LOW)
00064         end
00065         else begin
00066             `uvm_error("SCOREBOARD",
00067                 $sformatf("FAIL: ctrl=%0d rd1=%0h rd2=%0h EXPECTED result=%0h equal=%0b GOT result=%0h
00068                     equal=%0b",
00069                         data.i_alu_ctrl_EX, data.i_rd1_EX, data.i_rd2_EX,
00070                         exp_result, exp_equal, data.o_alu_result_EX, data.o_equal_EX))
00071             this.num_errors++;
00072         end
00073     endfunction
00074
00075     virtual function void check_phase(uvm_phase phase);
00076         super.check_phase(phase);
00077         if (this.num_errors > 0) begin
00078             `uvm_fatal("FINAL_RESULT",
00079                 $sformatf("TEST FAILED: Scoreboard found %0d errors.", num_errors))
00080         end
00081         else begin
00082             `uvm_info("FINAL_RESULT", "TEST PASS: All transactions were correct.", UVM_NONE)
00083         end
00084     endfunction
00085
00087     virtual task run_phase(uvm_phase phase);
00088         super.run_phase(phase);
00089     endtask
00090
00091 endclass

```

9.26 uvm/sequencer.sv File Reference

UVM sequencer that provides transactions to the driver.

This graph shows which files directly or indirectly include this file:



Classes

- class [sequencer](#)
UVM sequencer for ALU transactions.

9.26.1 Detailed Description

UVM sequencer that provides transactions to the driver.

Acts as the transaction source for the UVM driver in the ALU verification.

9.27 sequencer.sv

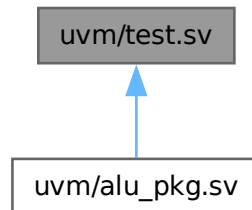
[Go to the documentation of this file.](#)

```
00001
00014 class sequencer extends uvm_sequencer #(pkt);
00015
00016     `uvm_component_utils (sequencer)
00017
00018     function new (string name="sequencer", uvm_component parent);
00019         super.new (name, parent);
00020     endfunction
00021
00022 endclass
```

9.28 uvm/test.sv File Reference

Top-level UVM test that builds the environment and starts the main sequence.

This graph shows which files directly or indirectly include this file:



Classes

- class `test`

Top-level UVM test for the ALU verification bench.

9.28.1 Detailed Description

Top-level UVM test that builds the environment and starts the main sequence.

Instantiates the verification environment and launches the ALU sequence that drives the DUT under test.

9.29 test.sv

[Go to the documentation of this file.](#)

```

00001
00016 class test extends uvm_test;
00017     `uvm_component_utils (test)
00018
00019     env m_env;
00020
00021     alu_sequence m_sequence;
00022
00023     function new (string name = "test", uvm_component parent = null);
00024         super.new (name, parent);
00025     endfunction
00026
00028     virtual function void build_phase (uvm_phase phase);
00029         super.build_phase (phase);
00030         m_env = env::type_id::create ("env", this);
00031     endfunction
00032
00034     virtual function void end_of_elaboration_phase (uvm_phase phase);
00035         uvm_root::get().print_topology();
00036     endfunction
00037
00039     virtual task run_phase(uvm_phase phase);
00040         super.run_phase(phase);
00041
00042         m_sequence = alu_sequence::type_id::create("sequence");
00043
00044         phase.raise_objection(this);
00045         m_sequence.start(m_env.m_agent.m_sequencer);
00046         phase.drop_objection(this);
00047     endtask
00048 endclass
  
```


Index

- [\\$unsigned](#)
 - [alu.v, 45](#)
- [a](#)
 - [alu.v, 45](#)
- [ADD](#)
 - [alu.v, 46](#)
- [adder.v](#)
 - [carry, 44](#)
 - [FAs_middle, 43](#)
 - [for, 43](#)
 - [i, 44](#)
 - [if, 43](#)
 - [internal_sum, 44](#)
 - [sum, 44](#)
- [adder_result](#)
 - [alu.v, 46](#)
- [agent, 17](#)
 - [build_phase, 18](#)
 - [connect_phase, 18](#)
 - [m_monitor, 19](#)
 - [m_sequencer, 19](#)
 - [new, 18](#)
 - [uvm_component_utils, 18](#)
- [alu.v](#)
 - [\\$unsigned, 45](#)
 - [a, 45](#)
 - [ADD, 46](#)
 - [adder_result, 46](#)
 - [AND, 46](#)
 - [AUIPC, 46](#)
 - [b, 45](#)
 - [begin, 46](#)
 - [BEQ, 46](#)
 - [BGE, 46](#)
 - [BGEU, 47](#)
 - [BLT, 47](#)
 - [BLTU, 47](#)
 - [BNE, 47](#)
 - [change_signal, 47](#)
 - [cin, 47](#)
 - [default, 47](#)
 - [EBREAK, 47](#)
 - [ECALL, 47](#)
 - [FENCE, 47](#)
 - [i_alu_ctrl_EX, 48](#)
 - [i_rd1_EX, 48](#)
 - [i_rd2_EX, 48](#)
 - [last_bit_equal, 48](#)
 - [LUI, 48](#)
 - [not_i_rd2_EX, 48](#)
 - [o_alu_result_EX, 48](#)
 - [o_equal_EX, 48](#)
 - [OR, 48](#)
 - [rd2_operand, 48](#)
 - [SLL, 49](#)
 - [SLT, 49](#)
 - [SLTU, 49](#)
 - [SRA, 49](#)
 - [SRL, 49](#)
 - [SUB, 49](#)
 - [sum, 46](#)
 - [WIDTH, 46](#)
 - [XOR, 49](#)
- [alu_ctrl_range](#)
 - [pkt, 31](#)
- [alu_golden](#)
 - [reference_model.c, 42](#)
 - [scoreboard.sv, 62](#)
- [alu_sequence, 19](#)
 - [body, 20](#)
 - [new, 20](#)
 - [num_packets, 21](#)
 - [uvm_object_utils, 20](#)
- [alu_wrapper.sv](#)
 - [always_ff, 50](#)
 - [i_rd1_EX, 50](#)
 - [i_rd2_EX, 50](#)
 - [o_alu_result_EX, 50](#)
 - [o_equal_EX, 50](#)
 - [U_ALU, 50](#)
- [always_ff](#)
 - [alu_wrapper.sv, 50](#)
- [AND](#)
 - [alu.v, 46](#)
- [ap_imp](#)
 - [scoreboard, 35](#)
- [AUIPC](#)
 - [alu.v, 46](#)
- [b](#)
 - [alu.v, 45](#)
- [begin](#)
 - [alu.v, 46](#)
- [BEQ](#)
 - [alu.v, 46](#)
- [BGE](#)
 - [alu.v, 46](#)
- [BGEU](#)
 - [alu.v, 47](#)

- BLT
 - alu.v, [47](#)
- BLTU
 - alu.v, [47](#)
- BNE
 - alu.v, [47](#)
- body
 - alu_sequence, [20](#)
- build_phase
 - agent, [18](#)
 - driver, [24](#)
 - env, [26](#)
 - monitor, [29](#)
 - scoreboard, [34](#)
 - test, [38](#)
- carry
 - adder.v, [44](#)
- cg_alu
 - coverage, [22](#)
- change_signal
 - alu.v, [47](#)
- check_phase
 - scoreboard, [34](#)
- cin
 - alu.v, [47](#)
- connect_phase
 - agent, [18](#)
 - env, [26](#)
- cout
 - full_adder.v, [51](#)
- coverage, [21](#)
 - cg_alu, [22](#)
 - new, [22](#)
 - uvm_component_utils, [22](#)
 - write, [22](#)
- default
 - alu.v, [47](#)
- drive_item
 - driver, [24](#)
- driver, [23](#)
 - build_phase, [24](#)
 - drive_item, [24](#)
 - new, [24](#)
 - req_pkt, [25](#)
 - run_phase, [24](#)
 - uvm_component_utils, [25](#)
- EBREAK
 - alu.v, [47](#)
- ECALL
 - alu.v, [47](#)
- end_of_elaboration_phase
 - test, [38](#)
- env, [25](#)
 - build_phase, [26](#)
 - connect_phase, [26](#)
 - m_coverage, [27](#)
 - m_scoreboard, [27](#)
 - new, [27](#)
 - uvm_component_utils, [27](#)
- FAs_middle
 - adder.v, [43](#)
- FENCE
 - alu.v, [47](#)
- filter Directory Reference, [15](#)
- filter/README.md, [39](#)
- for
 - adder.v, [43](#)
- forever
 - tb_top.sv, [52](#)
- full_adder.v
 - cout, [51](#)
 - sum, [51](#)
- i
 - adder.v, [44](#)
- i_alu_ctrl_EX
 - alu.v, [48](#)
 - pkt, [32](#)
- i_rd1_EX
 - alu.v, [48](#)
 - alu_wrapper.sv, [50](#)
 - pkt, [32](#)
- i_rd2_EX
 - alu.v, [48](#)
 - alu_wrapper.sv, [50](#)
 - pkt, [32](#)
- if
 - adder.v, [43](#)
- Interfaces, [12](#)
 - s_interface, [12](#)
- internal_sum
 - adder.v, [44](#)
- last_bit_equal
 - alu.v, [48](#)
- LUI
 - alu.v, [48](#)
- m_coverage
 - env, [27](#)
- m_monitor
 - agent, [19](#)
- m_scoreboard
 - env, [27](#)
- m_sequence
 - test, [38](#)
- m_sequencer
 - agent, [19](#)
- Modules, [11](#)
- mon_analysis_port
 - monitor, [29](#)
- monitor, [28](#)
 - build_phase, [29](#)
 - mon_analysis_port, [29](#)

- new, 29
- run_phase, 29
- uvm_component_utils, 29
- new
 - agent, 18
 - alu_sequence, 20
 - coverage, 22
 - driver, 24
 - env, 27
 - monitor, 29
 - pkt, 31
 - scoreboard, 34
 - test, 38
- not_i_rd2_EX
 - alu.v, 48
- num_errors
 - scoreboard, 35
- num_packets
 - alu_sequence, 21
- o_alu_result_EX
 - alu.v, 48
 - alu_wrapper.sv, 50
 - pkt, 32
- o_equal_EX
 - alu.v, 48
 - alu_wrapper.sv, 50
 - pkt, 32
- OP_ADD
 - reference_model.c, 40
- OP_AND
 - reference_model.c, 40
- OP_AUIPC
 - reference_model.c, 40
- OP_BEQ
 - reference_model.c, 40
- OP_BGE
 - reference_model.c, 40
- OP_BGEU
 - reference_model.c, 41
- OP_BLT
 - reference_model.c, 41
- OP_BLTU
 - reference_model.c, 41
- OP_BNE
 - reference_model.c, 41
- OP_EBREAK
 - reference_model.c, 41
- OP_ECALL
 - reference_model.c, 41
- OP_FENCE
 - reference_model.c, 41
- OP_LUI
 - reference_model.c, 41
- OP_OR
 - reference_model.c, 41
- OP_SLL
 - reference_model.c, 41
- OP_SLT
 - reference_model.c, 42
- OP_SLTU
 - reference_model.c, 42
- OP_SRA
 - reference_model.c, 42
- OP_SRL
 - reference_model.c, 42
- OP_SUB
 - reference_model.c, 42
- OP_XOR
 - reference_model.c, 42
- OR
 - alu.v, 48
- parent
 - sequencer, 36
- pkt, 30
 - alu_ctrl_range, 31
 - i_alu_ctrl_EX, 32
 - i_rd1_EX, 32
 - i_rd2_EX, 32
 - new, 31
 - o_alu_result_EX, 32
 - o_equal_EX, 32
 - uvm_field_int, 31
 - uvm_object_utils_begin, 31
- rd2_operand
 - alu.v, 48
- reference_model Directory Reference, 15
- reference_model.c
 - alu_golden, 42
 - OP_ADD, 40
 - OP_AND, 40
 - OP_AUIPC, 40
 - OP_BEQ, 40
 - OP_BGE, 40
 - OP_BGEU, 41
 - OP_BLT, 41
 - OP_BLTU, 41
 - OP_BNE, 41
 - OP_EBREAK, 41
 - OP_ECALL, 41
 - OP_FENCE, 41
 - OP_LUI, 41
 - OP_OR, 41
 - OP_SLL, 41
 - OP_SLT, 42
 - OP_SLTU, 42
 - OP_SRA, 42
 - OP_SRL, 42
 - OP_SUB, 42
 - OP_XOR, 42
- reference_model/reference_model.c, 39
- req_pkt
 - driver, 25
- rst
 - tb_top.sv, 52

- RTL design, 11
- rtl Directory Reference, 15
- rtl/adder.v, 43
- rtl/alu.v, 44
- rtl/alu_wrapper.sv, 49
- rtl/full_adder.v, 51
- run_phase
 - driver, 24
 - monitor, 29
 - scoreboard, 34
 - test, 38
- run_test
 - tb_top.sv, 52
- s_interface, 32
 - Interfaces, 12
- scoreboard, 33
 - ap_imp, 35
 - build_phase, 34
 - check_phase, 34
 - new, 34
 - num_errors, 35
 - run_phase, 34
 - uvm_component_utils, 34
 - write, 34
- scoreboard.sv
 - alu_golden, 62
- sequencer, 35
 - parent, 36
 - uvm_component_utils, 36
- SLL
 - alu.v, 49
- SLT
 - alu.v, 49
- SLTU
 - alu.v, 49
- SRA
 - alu.v, 49
- SRL
 - alu.v, 49
- SUB
 - alu.v, 49
- sum
 - adder.v, 44
 - alu.v, 46
 - full_adder.v, 51
- tb Directory Reference, 16
- tb/s_interface.sv, 51
- tb/tb_top.sv, 51
- tb_top.sv
 - forever, 52
 - rst, 52
 - run_test, 52
 - U_ALU_WRAPPER, 52
 - vif, 52
- test, 37
 - build_phase, 38
 - end_of_elaboration_phase, 38
 - m_sequence, 38
 - new, 38
 - run_phase, 38
 - uvm_component_utils, 38
- Testbench components, 11
- U_ALU
 - alu_wrapper.sv, 50
- U_ALU_WRAPPER
 - tb_top.sv, 52
- uvm Directory Reference, 16
- UVM verification components, 12
- uvm/agent.sv, 53
- uvm/alu_pkg.sv, 54
- uvm/alu_sequence.sv, 54, 55
- uvm/coverage.sv, 56
- uvm/driver.sv, 57, 58
- uvm/env.sv, 58, 59
- uvm/monitor.sv, 59, 60
- uvm/pkt.sv, 61
- uvm/scoreboard.sv, 62, 63
- uvm/sequencer.sv, 64
- uvm/test.sv, 65
- uvm_component_utils
 - agent, 18
 - coverage, 22
 - driver, 25
 - env, 27
 - monitor, 29
 - scoreboard, 34
 - sequencer, 36
 - test, 38
- uvm_field_int
 - pkt, 31
- uvm_object_utils
 - alu_sequence, 20
- uvm_object_utils_begin
 - pkt, 31
- vif
 - tb_top.sv, 52
- WIDTH
 - alu.v, 46
- write
 - coverage, 22
 - scoreboard, 34
- XOR
 - alu.v, 49